

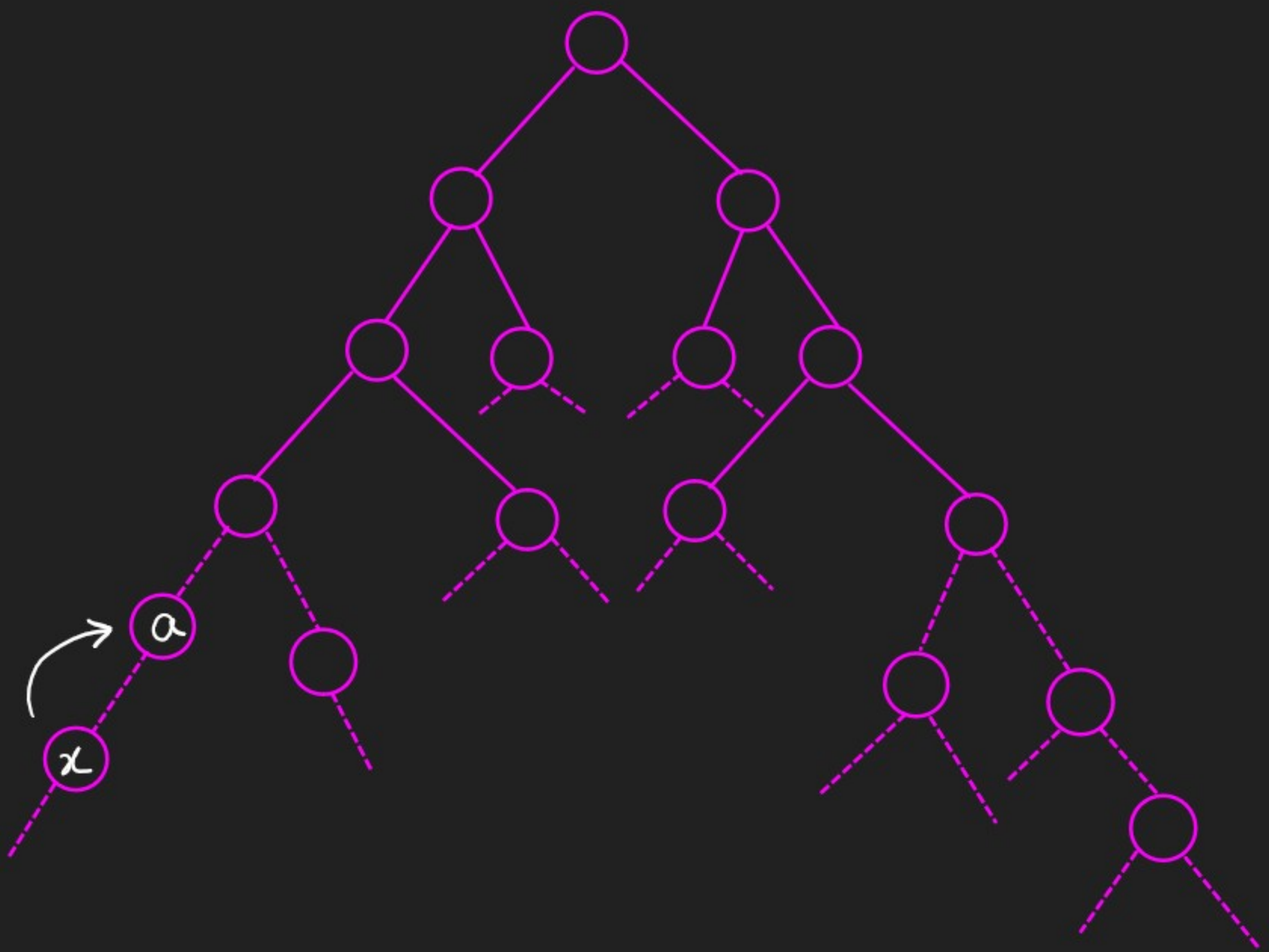
leetcode pattern

Binary lifting

Introduction:

what? why? How?

Suppose I have a tree of ~2 lakh node



- what is ancestor of x ? // a
- what is 50th ancestor of x ? // b
- what is 80th ancestor of x ? // c

That's means:

k^{th} ancestor of node x

- Time complexity: $O(k)$ per query

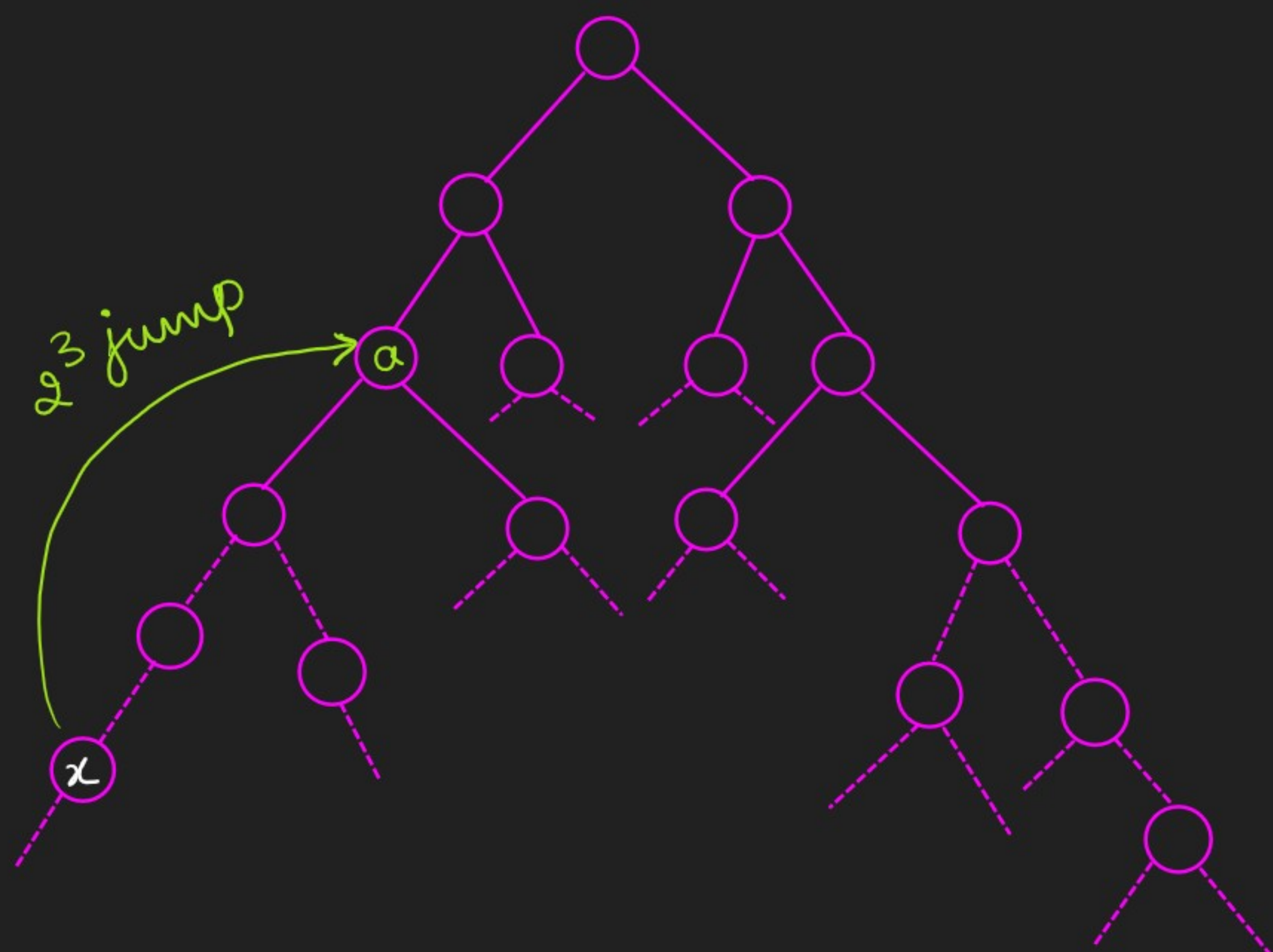
very slow solⁿ

Binary lifting

- It helps to jump upward in the tree very efficiently.

→ Jump one step at a time. ✗

→ climb in larger jump. ✓



- Jump in power of 2.

i.e. $2^0, 2^1, 2^2, 2^3, 2^4, \dots$

why power of 2 only?

$$\begin{aligned}
 x \rightarrow 14 \text{ jump} &= \overbrace{2^3 + 2^2 + 2^1}^{\text{unique value}} \\
 &= 8 + 4 + 2 \\
 &= 14 \\
 &= 3^3 + 3^1 + \overbrace{3^0 + 3^0}^{\text{duplicate value}} \\
 &= 9 + 3 + 1 + 1 \\
 &= 14
 \end{aligned}$$

* Every number can be expressed as a unique power of 2.

14 Binary →

1	1	1	0
↓	↓	↓	↓
2^3	2^2	2^1	2^0
↓	↓	↓	↓
8	4	2	1

} Here Binary representation tell us, how many take jump, and which one not take jump.

13 Binary → $2^3 + 2^2 + 2^1$

↓

1	1	0	1
↓	↓	↓	↓
2^3	2^2	2^1	2^0

↑
नहीं लेना है

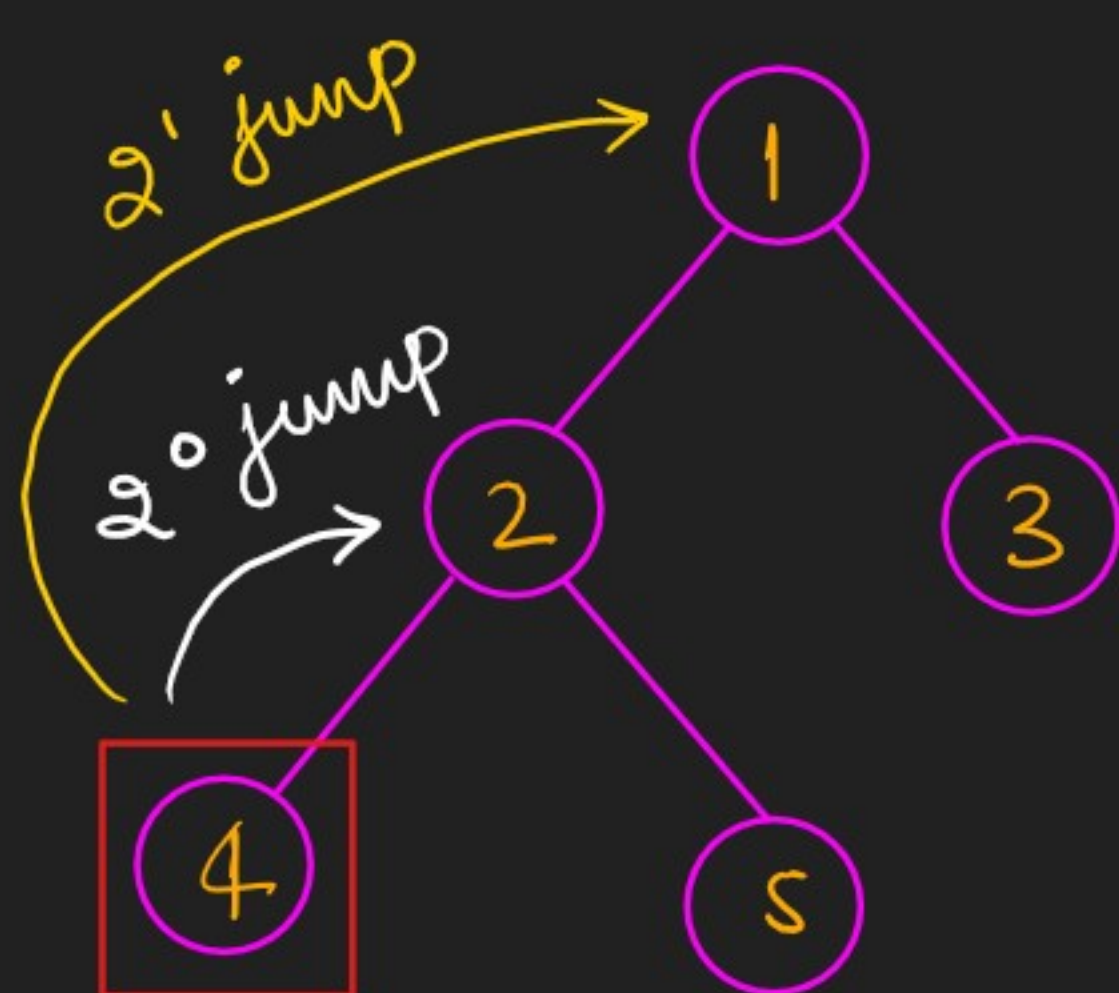
that's why

we called

Binary lifting

How to store power of 2 jump information

$x \rightarrow 2^i \Rightarrow$ which ancestor ???



$4 \rightarrow 2^{(0)^{\text{power}}} \rightarrow 2$
node jump ancestor

$4 \rightarrow 2^{(1)^{\text{power}}} \rightarrow 1$
node jump ancestor

how to
store it
???

$\text{parent}[\text{node}][j] = 2^{j^{\text{th}}}$ ancestor node

if I take 2^j jump from node, which ancestor I will reach.

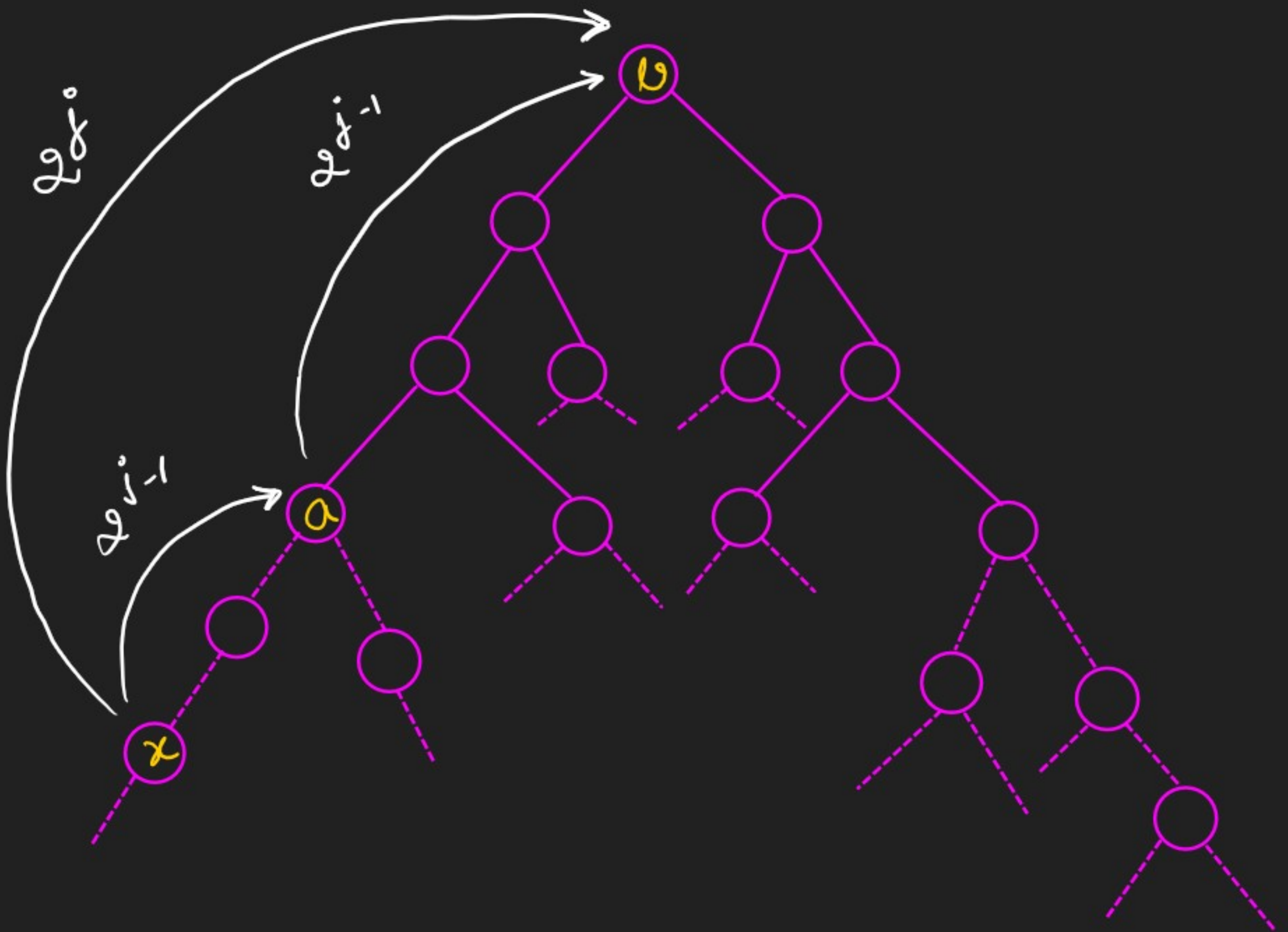
i.e. $\text{parent}[4][0] = 2$

$\text{parent}[4][1] = 1$

$j=0$
• $\text{parent}[\text{node}][0] = 2^{0^{\text{th}}}$ ancestor node = 1 jump up

$j=1$
• $\text{parent}[\text{node}][1] = 2^{1^{\text{th}}}$ ancestor node = 2 jump up

The only thing to keep in mind 



node $2^j \rightarrow$ ancestor ??

$\text{parent}[\text{node}][j] = ??$

$$2^j = 2^{j-1} + 2^{j-1}$$

$$a = \text{parent}[\text{node}][j-1]$$

$$b = \text{parent}[a][j-1]$$

↑
source node

$$\text{parent}[\text{node}][j] = \text{parent}[a][j-1]$$

Replace it

$$\text{parent}[\text{parent}[a][j-1]][j-1]$$

$$\text{parent}[\text{node}][j] = \text{parent}[\text{parent}[a][j-1]][j-1]$$

Most Important Recurrence Relation.

$$\text{parent}[\text{node}][j] \begin{cases} \rightarrow \text{Row} = \text{nodes} \\ \rightarrow \text{col} = \log(n) \end{cases}$$

Code Snippet

// Building the Ancestor Table

```
for(j=0; j < log2(n) + 1; j++) {  
    for(node=0; node < n; node++) {  
        if (parent[node][j-1] != -1) {  
            parent[node][j] = parent[parent[node]  
                [j-1]][j-1];  
        }  
    }  
}
```